

RoboMaster ENTERPRIZE

Embedded Tutorial 1

*Introduction to
Embedded System*



讲师介绍



RM2027嵌入式组长
彭晨耕



RM2027嵌入式成员
李睿米



RM2027项管
RM2026副队长&算法组长
杨静悠

讲师介绍



RM2026步兵嵌入式
郭彦钦



RM2026英雄嵌入式
潘俊天



RM2026哨兵嵌入式
谢云帆

TA介绍



RM2026嵌入式组长
王颢竣 Ronin



RM2026飞镖嵌入式
郑斯阳



RM2027算法组长
黄俊霖 Nicholas

TA介绍



RM2027算法成员
肖志昊 Akano



RM2027算法成员
李雪阳 Somo



RM2026步兵嵌入式
高海东

TA介绍



RM2026嵌入式成员
冯美维



RM2026哨兵算法
董天扬

从这节课开始.....

- 集中于**嵌入式系统**和控制算法
- 所有主题都会在**Internal**中起作用
- 没有C++基础的同学可以根据**面试通过文件**中的C++课程进行学习,与我们所有课程目的并非在十分有限的时间内教会你所有底层的细节知识,而是提供单片机实际应用时所需要的知识和方向.
- **WARNING: 不要把开发板放在金属上面, 会导致短路!!!**

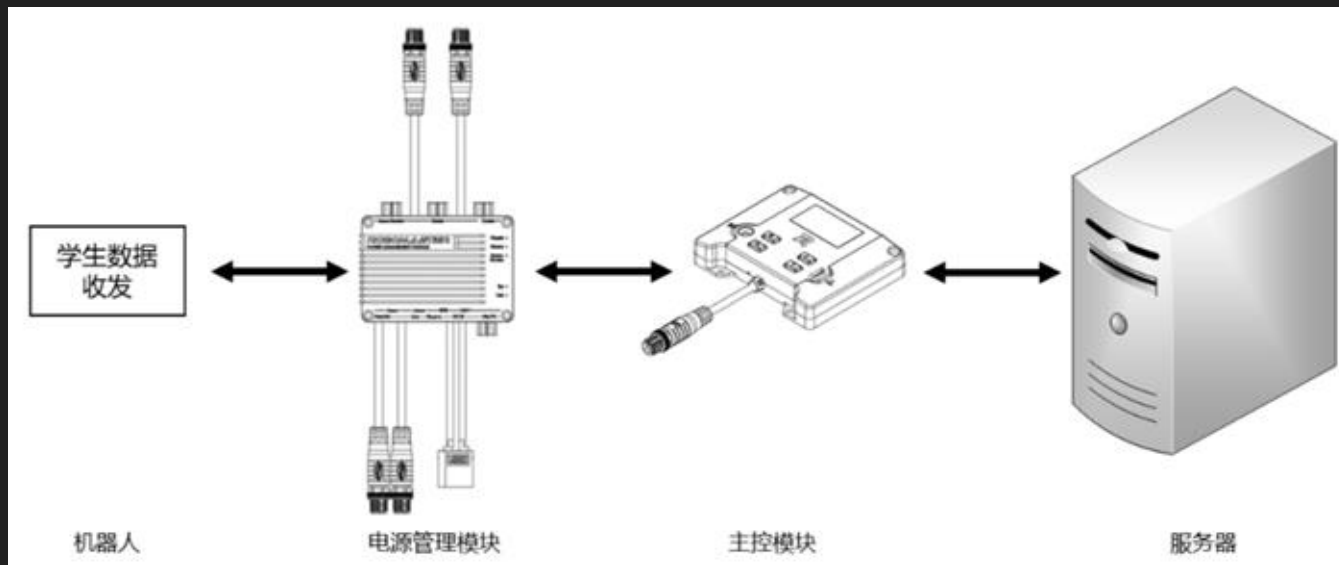
Tuto课程流程

1：嵌入式引入 & GPIO



Tuto课程流程

2 : UART(串口通信)



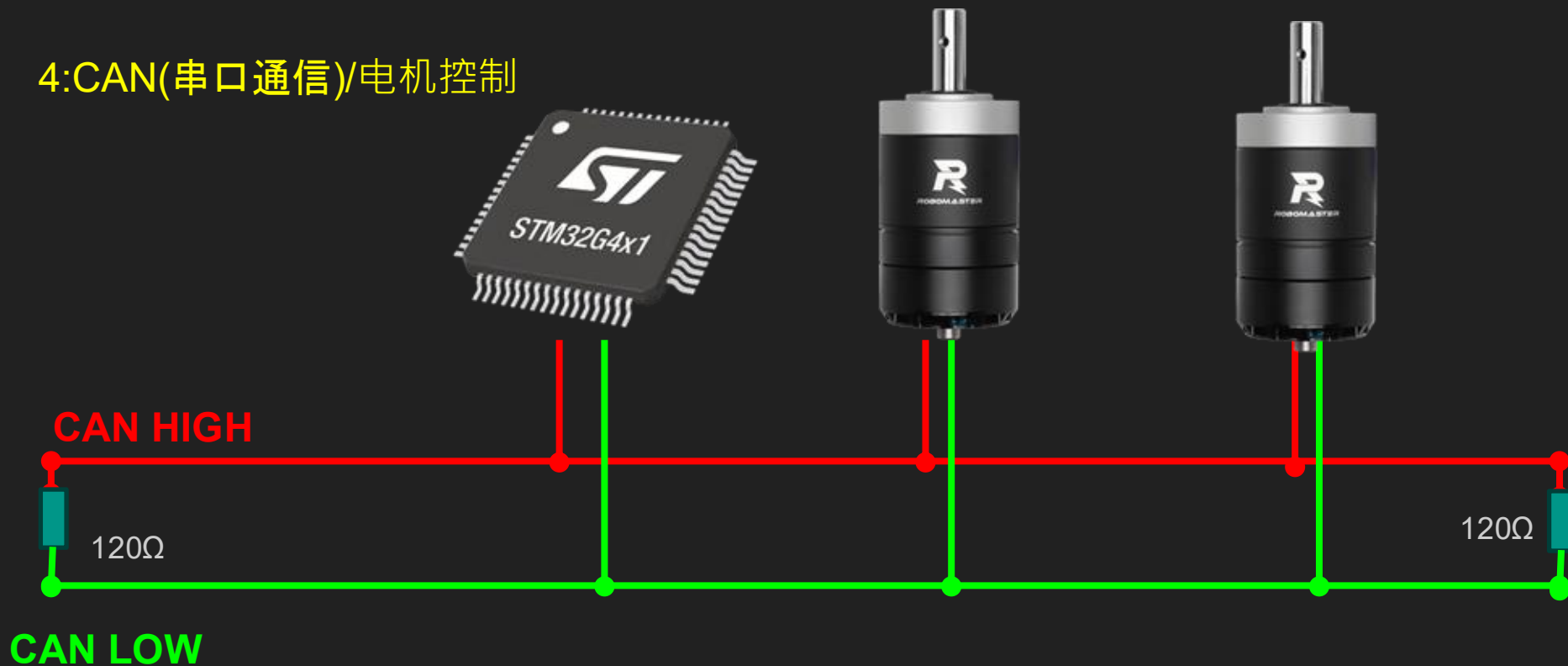
Tuto课程流程

3:Interrupt(中断)



Tuto课程流程

4:CAN(串口通信)/电机控制



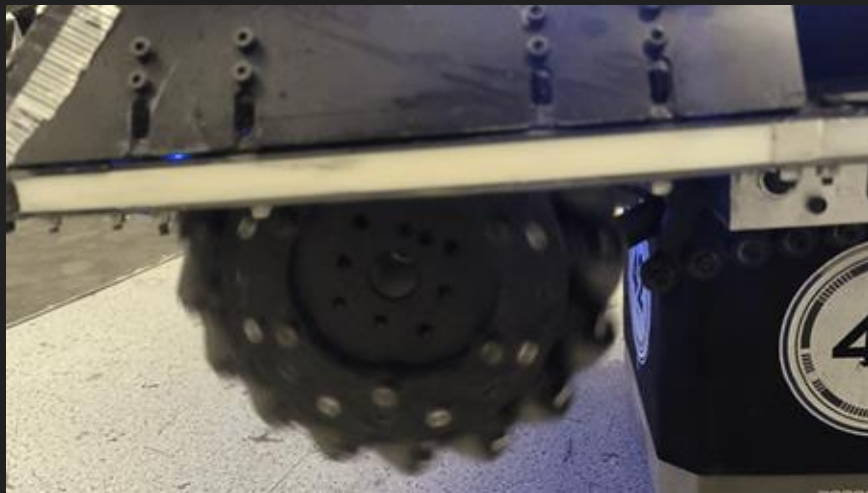
Tuto课程流程

5:PID控制器

控制云台的角度



控制轮电机转速



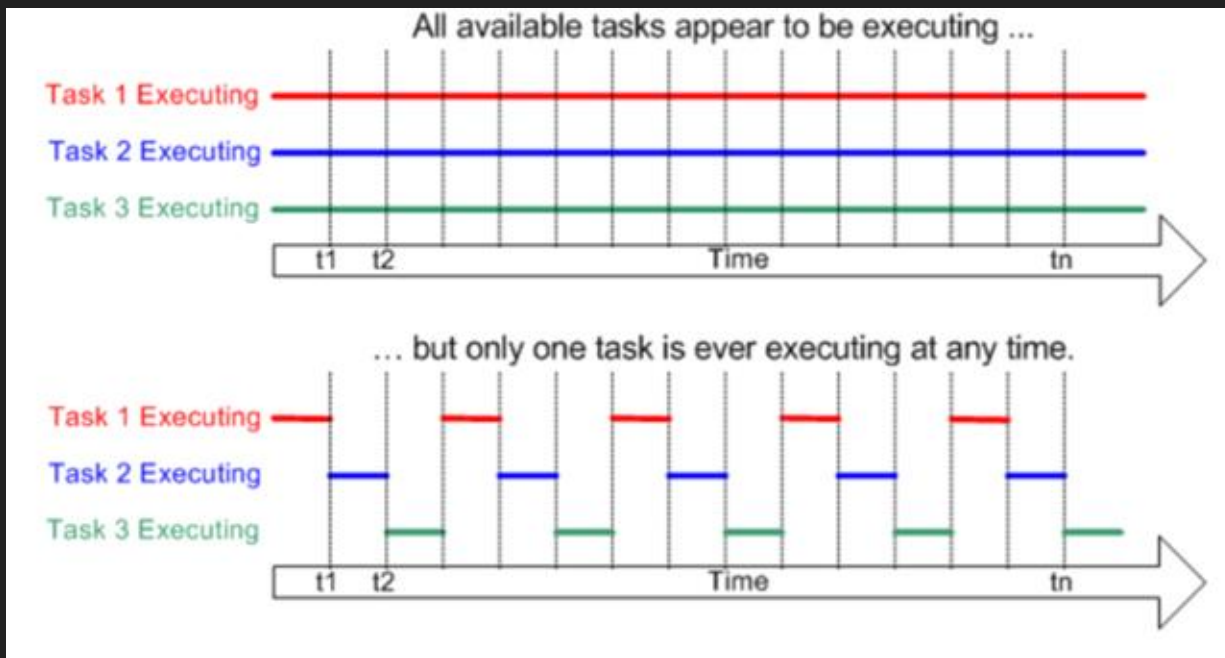
Tuto课程流程

6：定时器/PWM



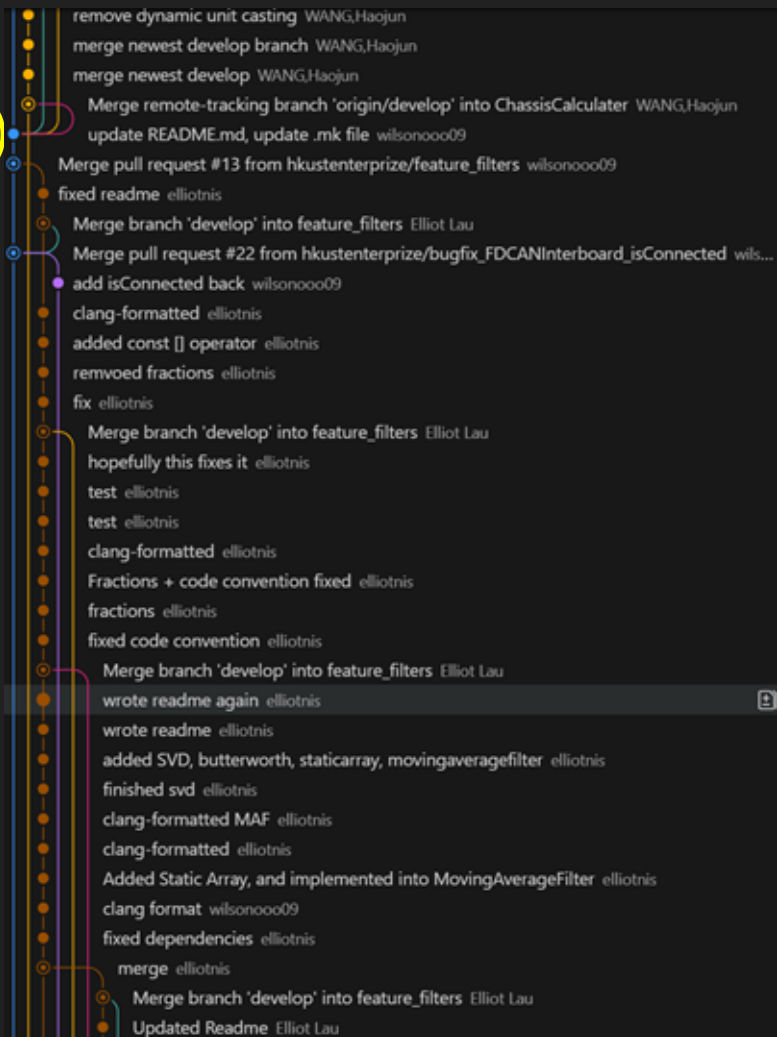
Tuto课程流程（录播）

7:FreeRTOS(Free Real-Time Operating System)



Tuto课程流程 (录播)

8:Git-分布式版本控制



内容

- 开发板/开发流程介绍
- 如何使用STM32CubeMX创建工程
- 如何使用Makefile进行编译
- 如何使用Ozone进行烧录
- 使用GPIO点灯
- 如何使用Ozone进行调试

嵌入式工作overview

基础硬件外设
驱动控制

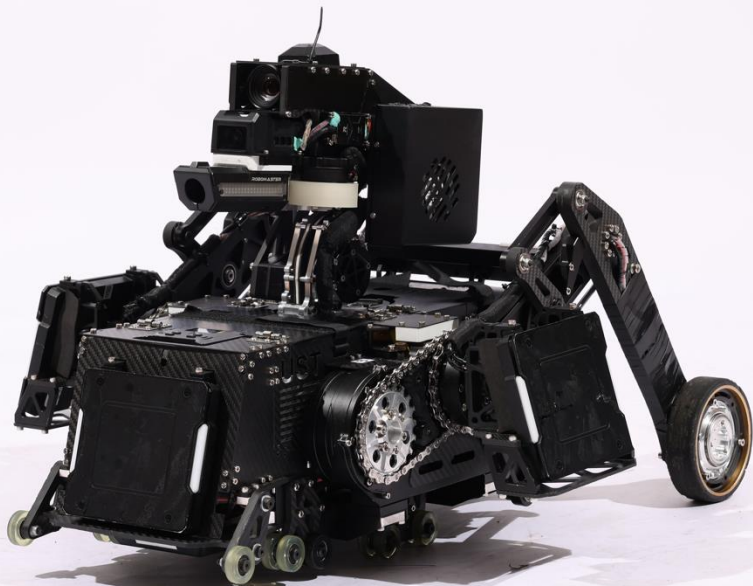
CAN通讯

UART通讯

SPI通讯

.....

嵌入式工作overview



人运动控
逻辑

单轮底盘

双轮底盘

机械臂



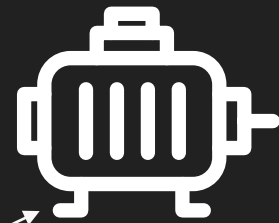
What is embedded system?

嵌入式系统 = MCU + 外设



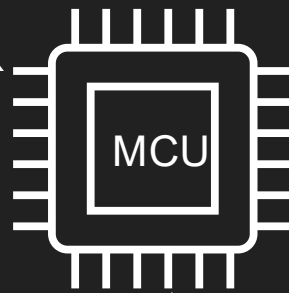
遥控器

UART通讯

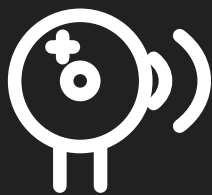


电机

CAN 通讯

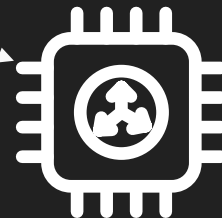


PWM信号



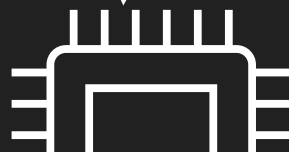
蜂鸣器

SPI 通讯



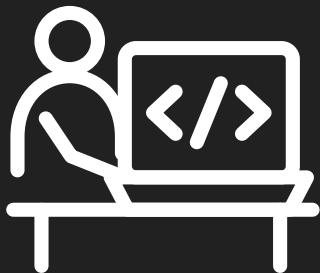
IMU

板间通讯

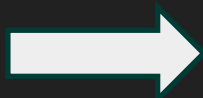


开发流程

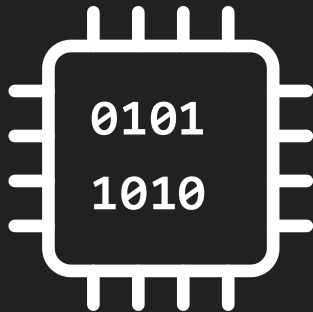
编写C语言源码



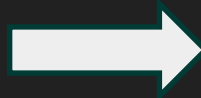
编译



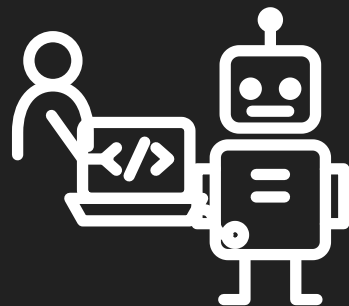
二进制机器码



烧录



调试



名词解释:

- 单片机
- 引脚
- 编译
- 烧录
- 调试



一些基础概念

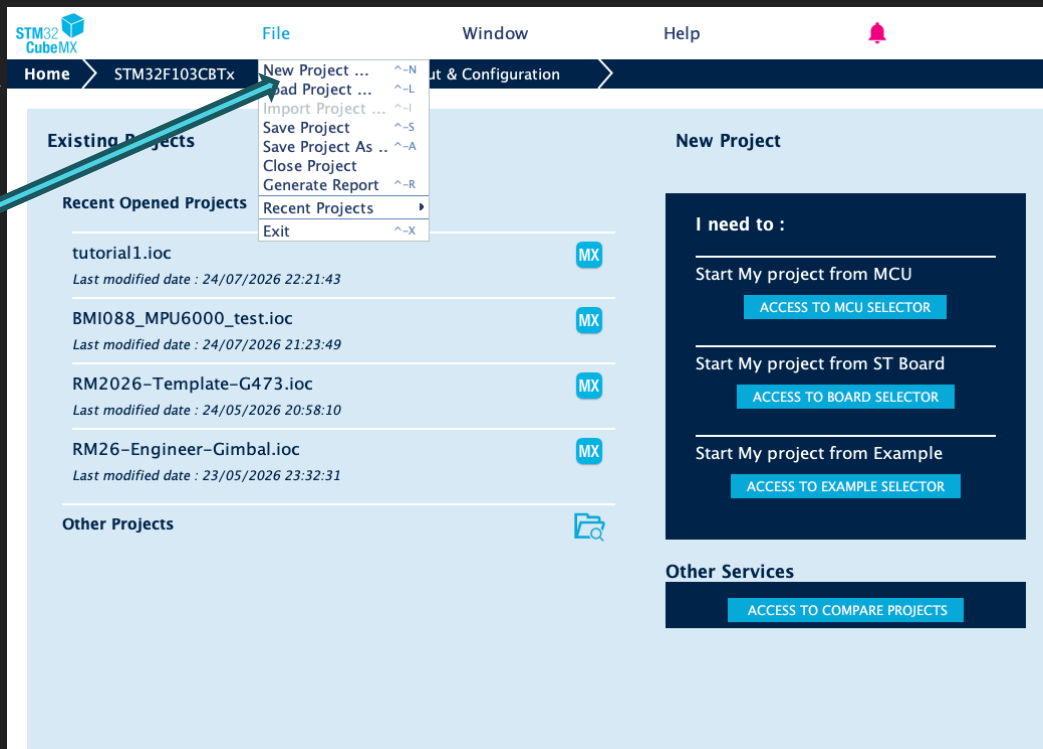
1. 我们的代码最后是跑在单片机上的，而不是电脑上
2. 我们可以通过Ozone软件去读取单片机上的一些变量的值，进行debug

如何使用STM32CubeMX创建？

1. 选择目标microcontroller (STM32F103CBT6)
2. 在 System Core, SYS, 选择debug 为 Serial Wire
3. 在 System Core, RCC. 选择 HSE 为 Crystal/Ceramic Resonator
4. 在 Clock Configuration 设置 INPUT FREQUENCY 为 16MHz, PLL Source Mux 选择HSE, HCLK键入72 MHz
5. 在Project Manager tab (Generate Code下面), 选择 Makefile 作为 toolchain/IDE
6. 在 Code Generator(left), 选择 “Generate peripheral initialization as a pair of ‘.c/.h’ files per peripheral”
7. 点击GENERATE CODE 生成代码.

第一步: 创建New Project

- Reminder: 记得登录ST账号
- 选择File->New Project



第二步: 选择MCU

- 左上角输入MCU型号:

STM32F103CBT6

- 选择STM32F103CB

The screenshot shows the STM32CubeMX MCU/MPU Selector interface. The left sidebar contains filters for Commercial, F103CBT, and various product info categories. The main area displays the STM32F103CBT6 product details, including its features and a table of available MCUs/MPUs.

PRODUCT INFO

- Segment
- Series
- Package
- Core
- Coprocessor

MEMORY

- Flash = 128 (kBytes)
- EEPROM = 0 (Bytes)
- RAM Total = 20 (kBytes)
- RAM = 20 (kBytes)
- CCM RAM = 0 (kBytes)
- On-chip SRAM = 0 (kBytes)
- ITCM/DTCM RAM
- DRAM Support
- Flash Support

STM32F1 Series

STM32F103CBT6 Mainstream Performance line, Arm Cortex-M3 MCU with 128 Kbytes of Flash

Unit Price for 10KU (US\$) : 3.2762

The STM32F103xx medium-density performance line family incorporates the high-performance Arm® Cortex®-M3 32-bit RISC core operating at up to 128 Kbytes and SRAM up to 20 Kbytes, and an extensive range of enhanced I/Os and peripherals connected to two APB buses. All devices contain a timer, as well as standard and advanced communication interfaces: up to two I²Cs and SPIs, three USARTs, a USB and a CAN.

The devices operate from a 2.0 to 3.6 V power supply. They are available in both the -40 to +85°C temperature range and the -40 to +105 °C extended temperature range, which allows the design of low-power applications.

The STM32F103xx medium-density performance line family includes devices in six different package types: from 36 pins to 100 pins. Depending on the package type, the description below gives an overview of the complete range of peripherals proposed in this family.

These features make the STM32F103xx medium-density performance line microcontroller family suitable for a wide range of applications such as automotive, industrial and gaming peripherals, GPS platforms, industrial applications, PLCs, inverters, printers, scanners, alarm systems, video intercoms, and HVACs.

Features

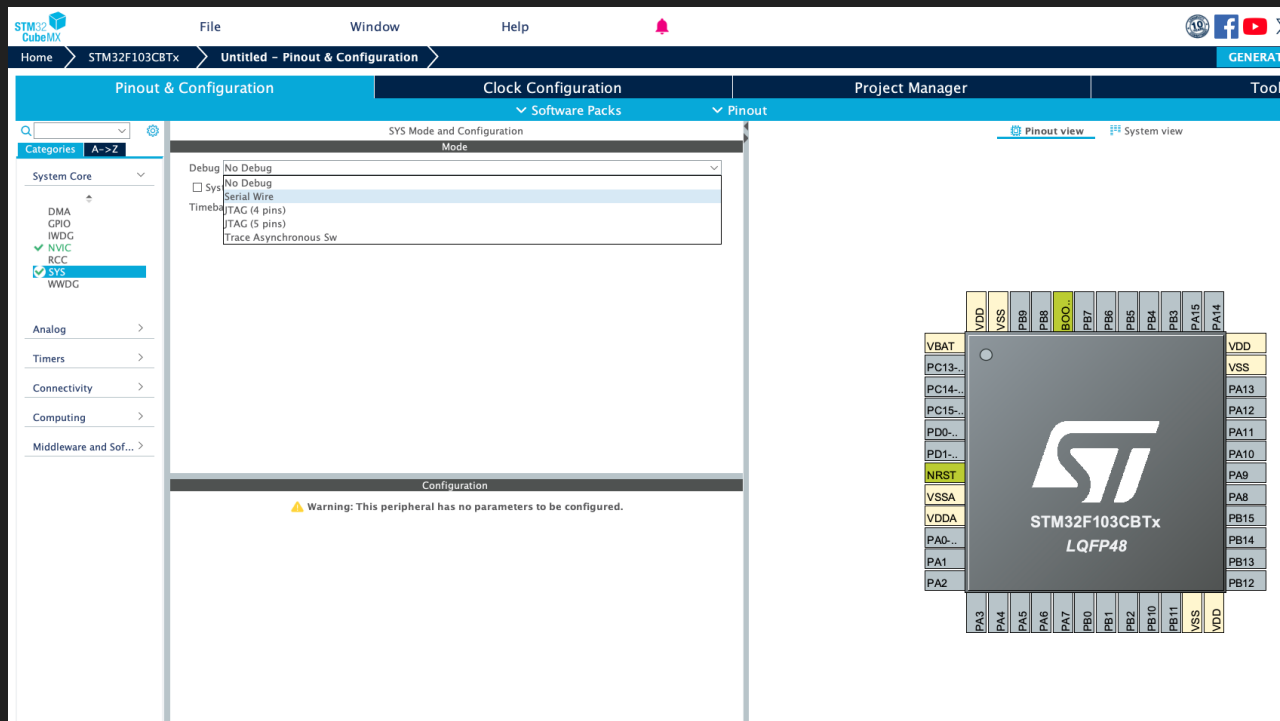
- Arm® 32-bit Cortex®-M3 CPU core

MCUs/MPUs List: 4 items

	Commercial Part No	Package	Reference	Marketing Status	Unit Price for 10KU (US\$)	Board	
☆	STM32F103CBT6		STM32F103CBTx	Active	3.2762		LQFP 48 7x7
☆	STM32F103CBT6TR		STM32F103CBTx	Active	3.2762		LQFP 48 7x7
☆	STM32F103CBT7		STM32F103CBTx	Active	3.5055		LQFP 48 7x7
☆	STM32F103CBT7TR		STM32F103CBTx	Active	3.5055		LQFP 48 7x7

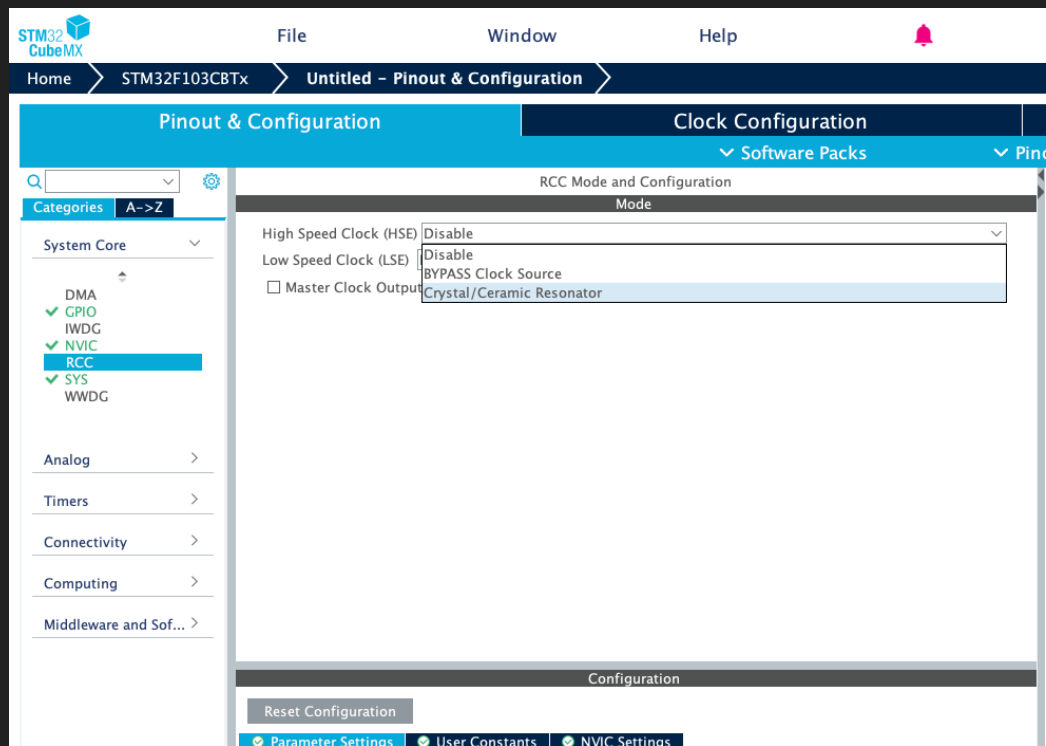
第三步: 设置SYS

- 选择System Core->SYS->Debug->Serial Wire



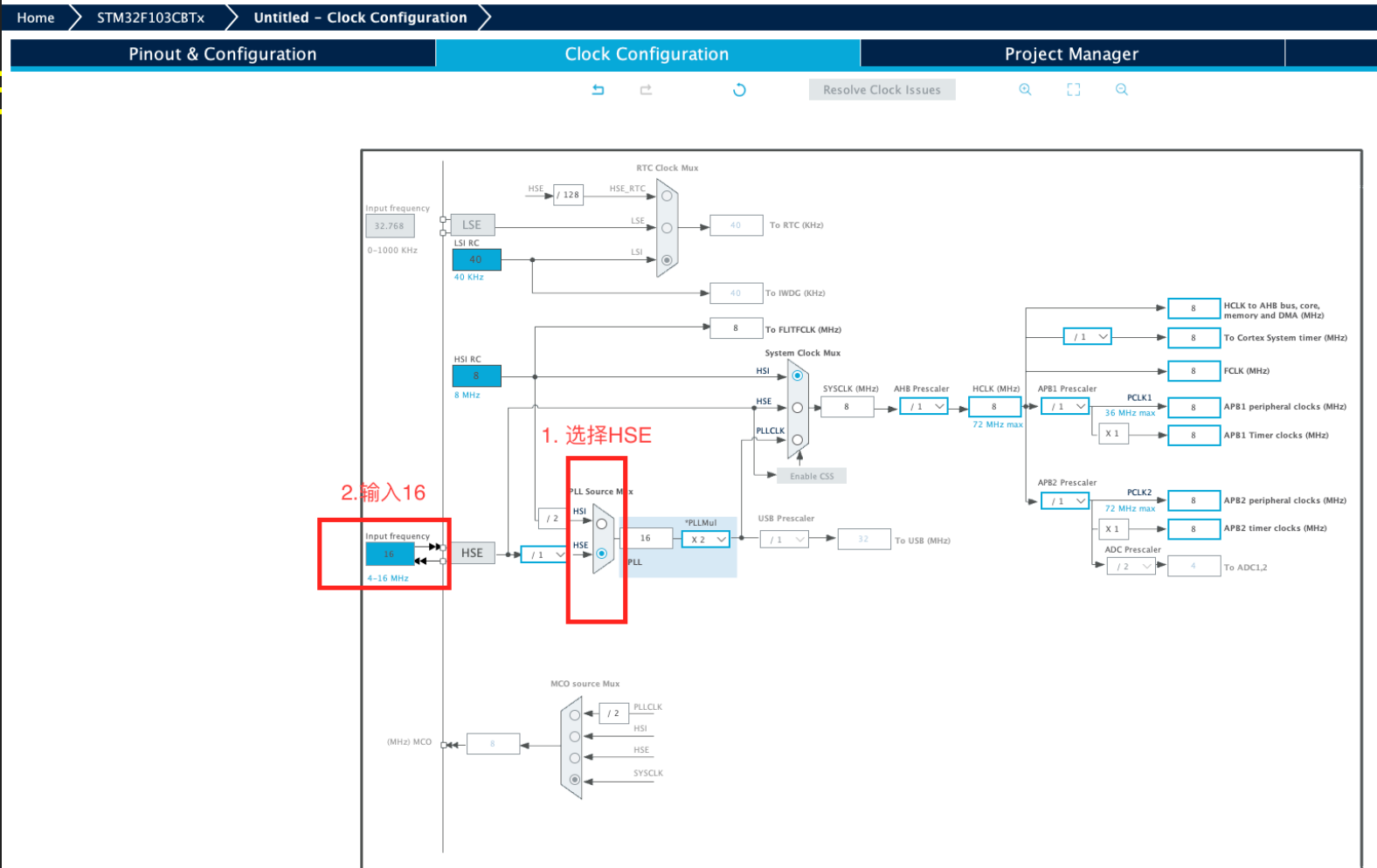
第四步: 设置RCC

- 选择RCC->High Speed Clock (HSE)-> Crystal/Ceramic Resonator



第五步: 设置时钟

- 选择Clock Configuration -> Input frequency - 16HZ, PLL Source Mux - HSE

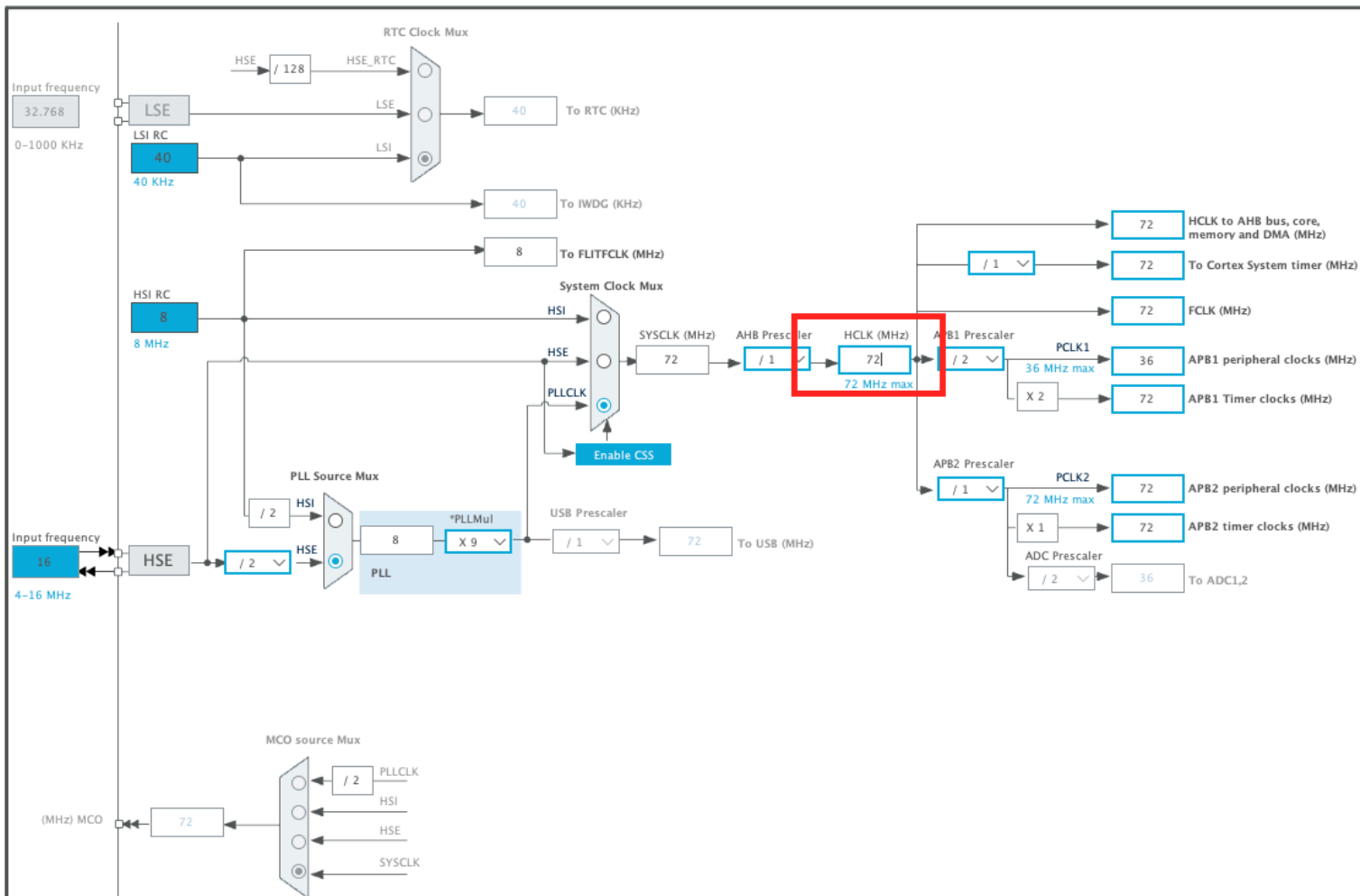


第五步: 设置时钟

HCLK(MHz) -> 72MHz, 回车confirm

第五

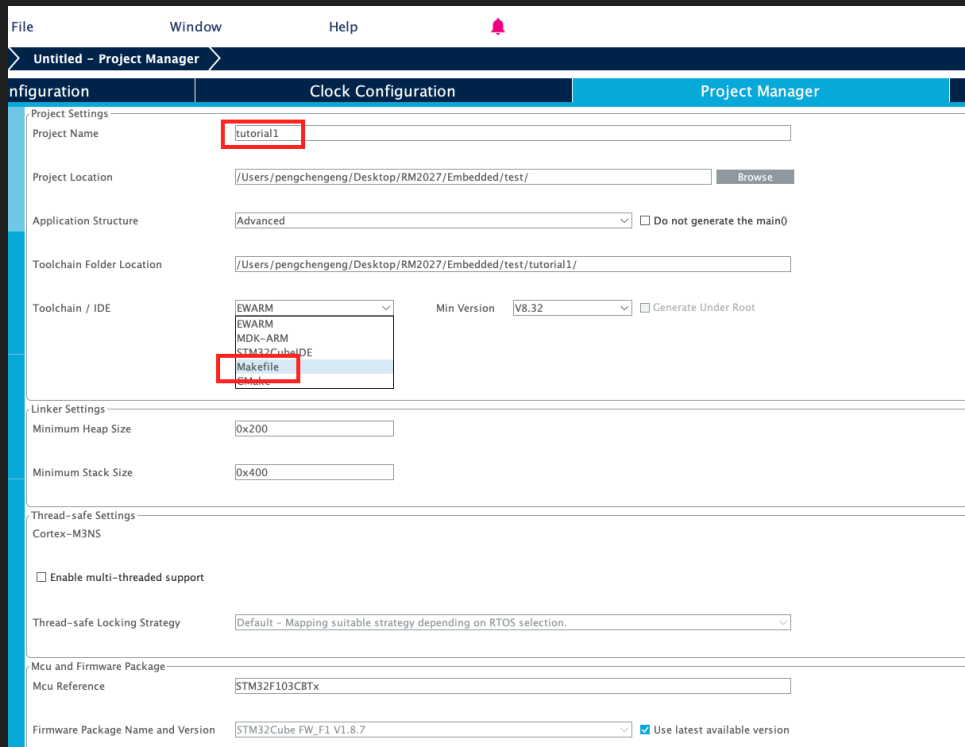
HCL



第六步: 选择Makefile作为工具链

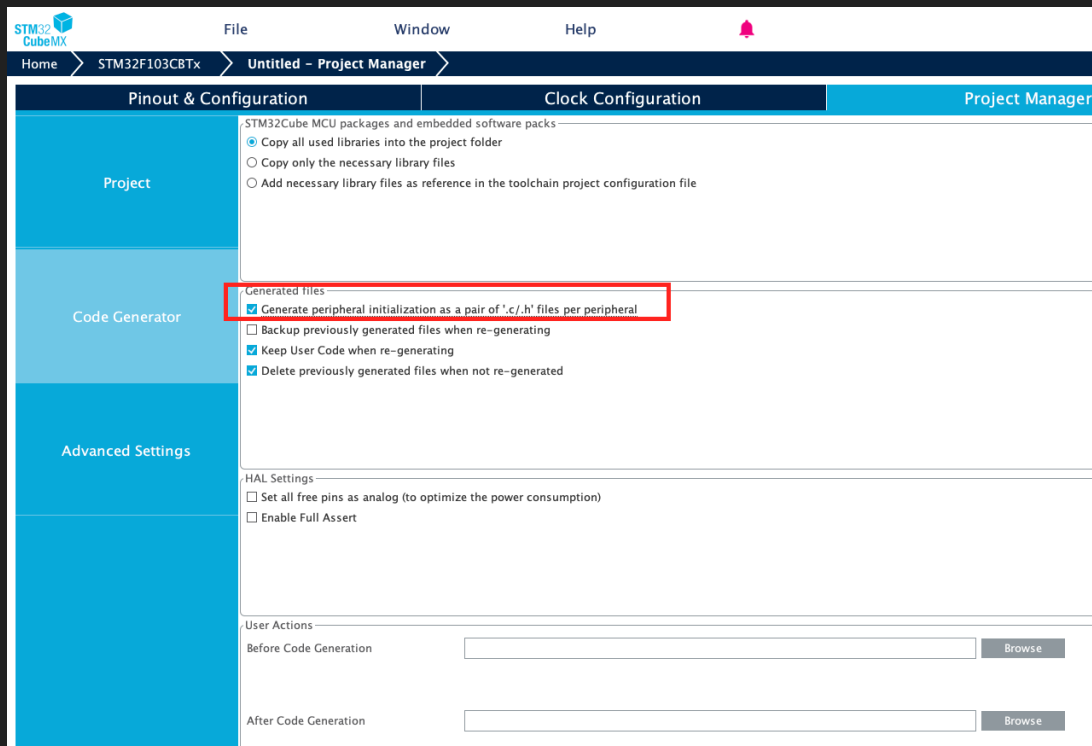
- 选择 Project Manager-> 输入project name -> 选择 Toolchain/IDE为Makefile.

路径不要选择在
带有**空格**和**中文**
的文件夹中!!!!!!



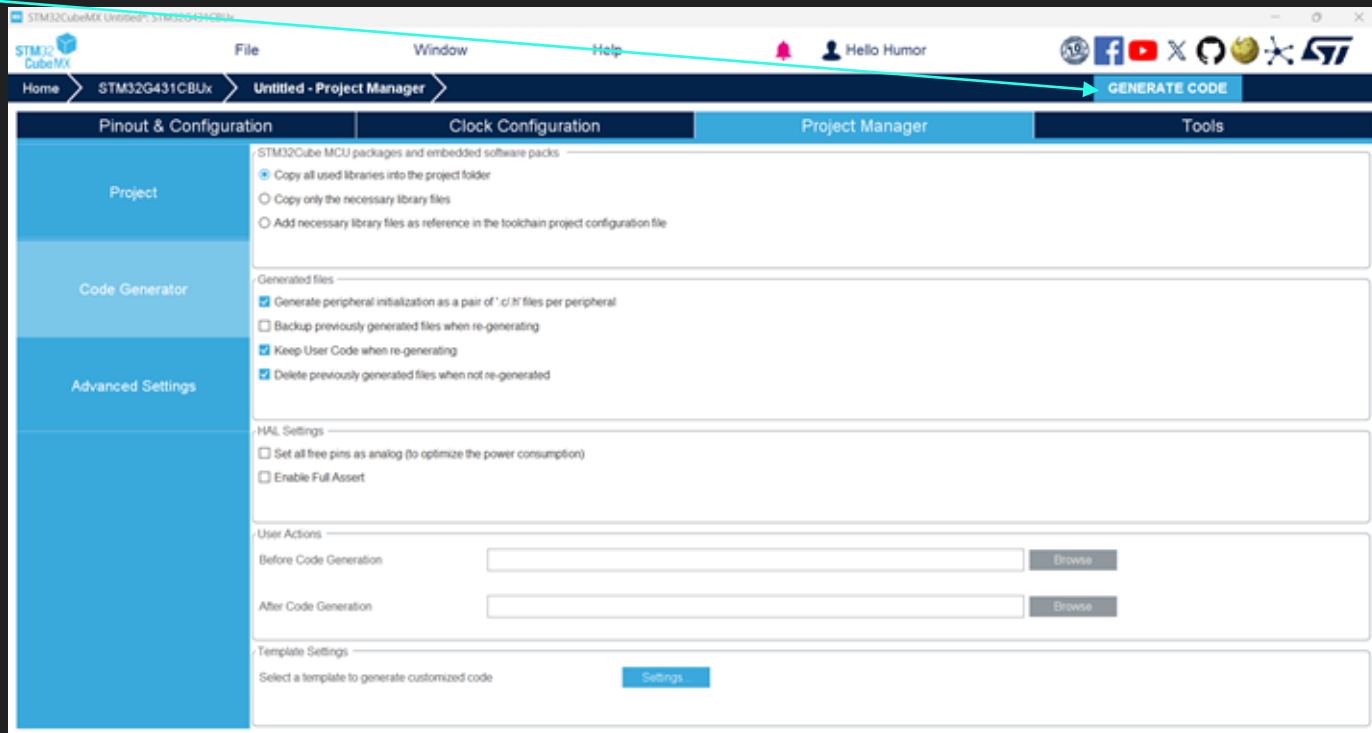
第七步: 修改generate类型

- 选择 Code Generator -> 点击 Generate peripheral initialization as a pair.....



第八步: 生成代码

- 点击 GENERATE CODE



点灯

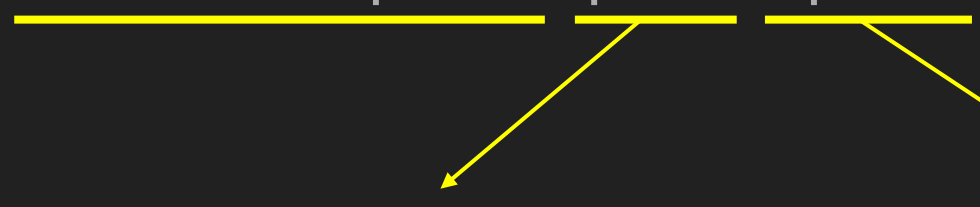
- 嵌入式工程师第一步：点亮LED小灯



General Purpose In/Output - GPIO

GPIO – 通用输入输出引脚

General **P**urpose **I**nput **O**utput



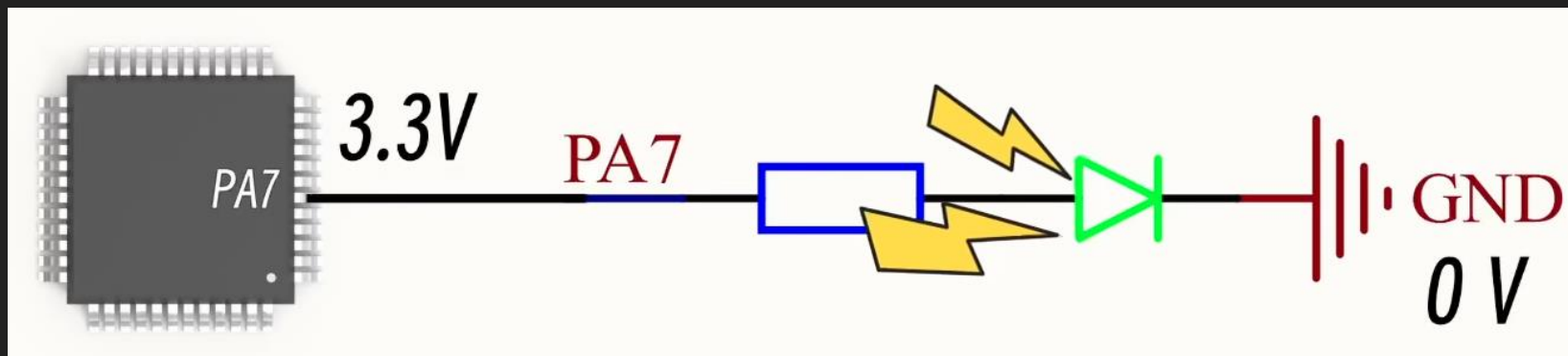
```
graph TD; GPIO[GPIO] --> Input[输入]; GPIO --> Output[输出];
```

输入（外界to芯片）
检测某个引脚的电压

输出（芯片to外界）
向某个引脚输出电压

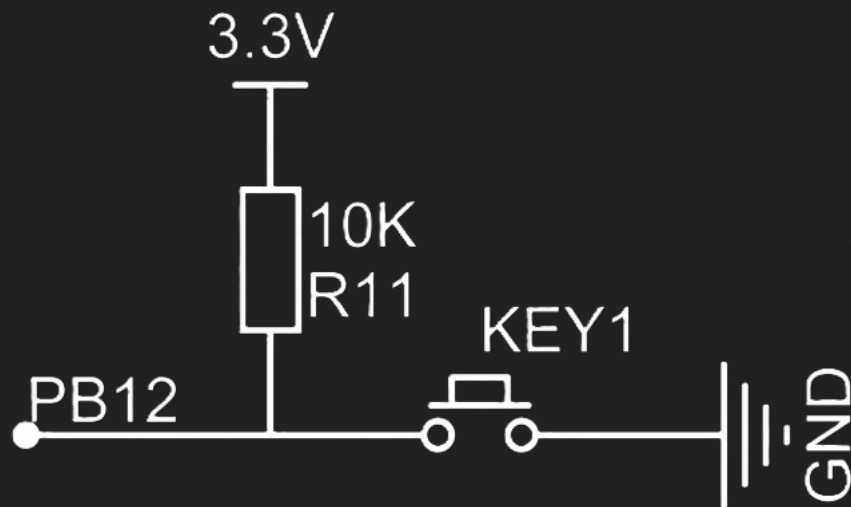
使用场景

Output Mode : 点亮LED小灯



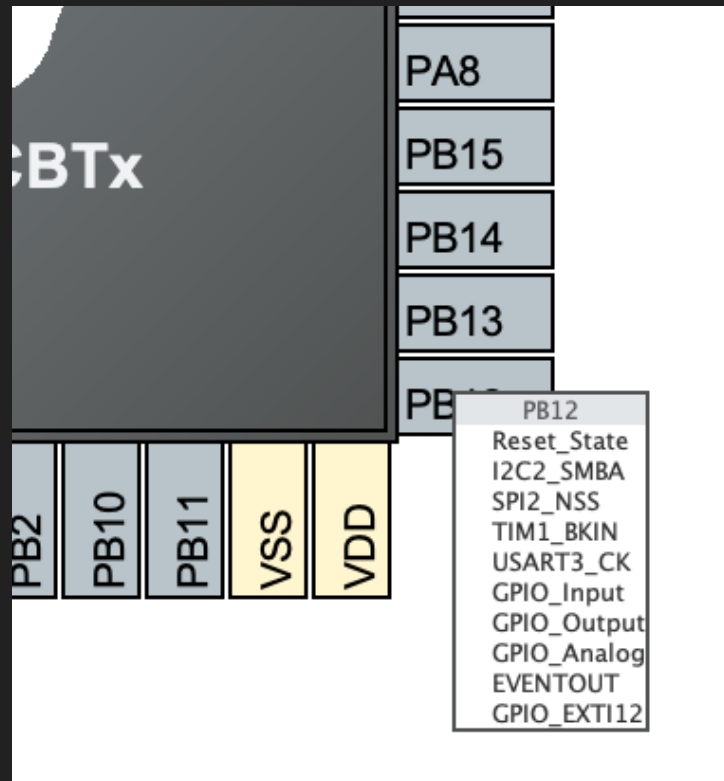
使用场景

Input Mode : 按键检测

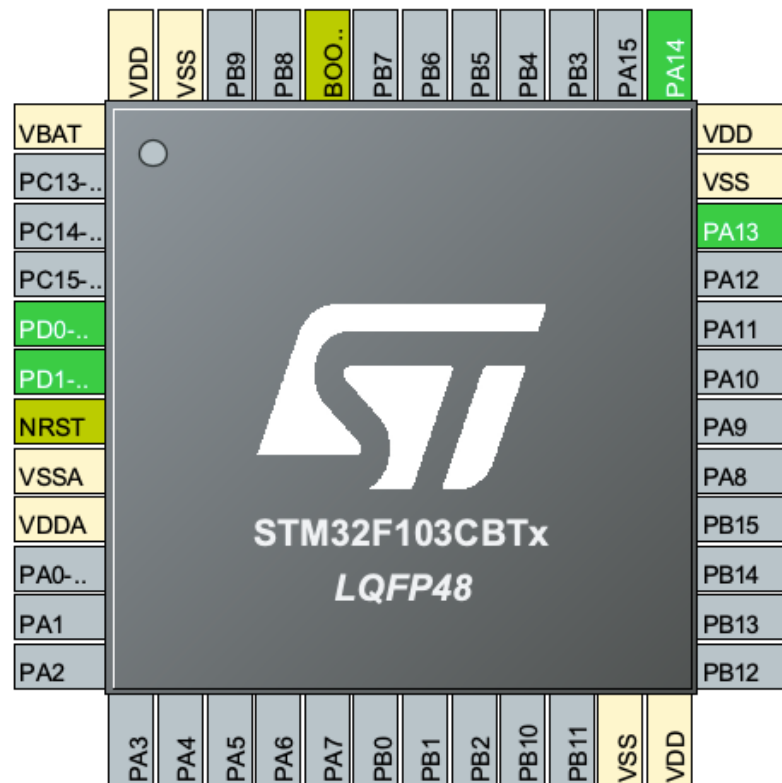


用 STM32CubeMX 配置 GPIO

1. 选择一个引脚(pin),比如PB12
2. 选择 GPIO_Input 作为 input mode
3. 选择 GPIO_Output 作为 output mode

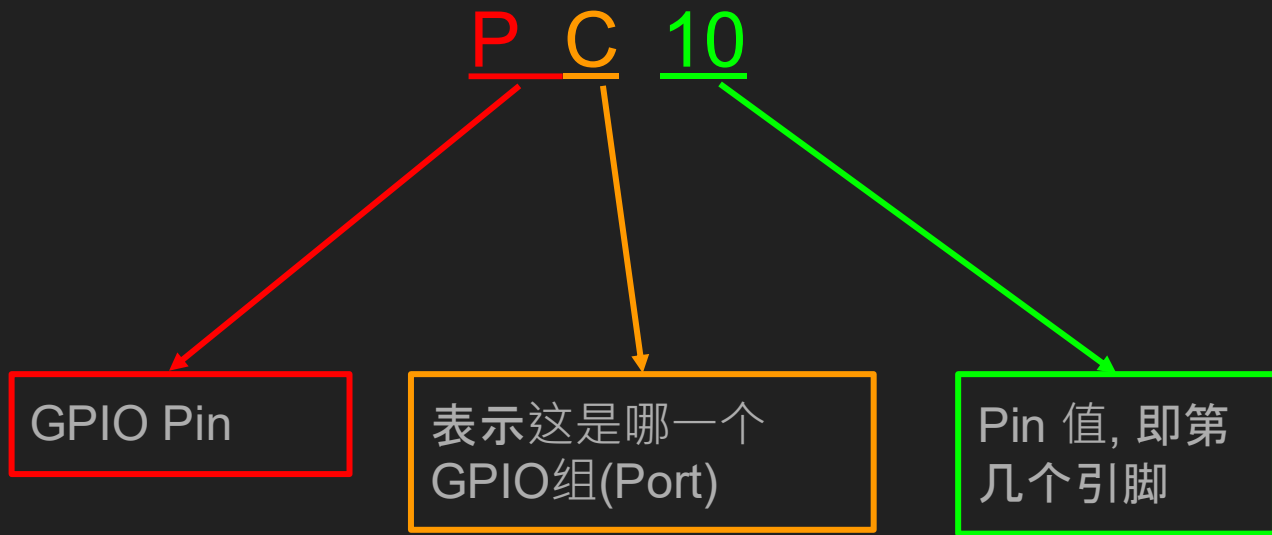


RCC_OSC_IN
RCC_OSC_OUT



SYS_JTCK

SYS_JTMS-SWDIO



这个GPIO的Port和Pin各是什么??

PA0

代码怎么写？？

Input Mode (GPIO输入)

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
```

Input Mode (GPIO输入)

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
```

返回值类型：这个GPIO引脚的状态

可能的值：高电平/低电平

Input Mode (GPIO输入)

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
```

函数名：读GPIO引脚

HAL是一个代码库，代表这个函数来自HAL库

Input Mode (GPIO输入)

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
```

参数1：这个引脚的组 (port)

参数2：这个引脚的pin值

Input Mode (GPIO输入)

示例：

```
GPIO_PinState pin_state = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_12);
```

代表读PB12的状态

Output Mode (GPIO输出)

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
```

无返回值

Output Mode (GPIO输出)

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
```

写某个pin

Output Mode (GPIO输出)

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
```

参数1：要写的引脚的组

参数2：要写的引脚的pin

参数3：要写的引脚状态

GPIO 标签

给GPIO增加一个标签使其更加易读

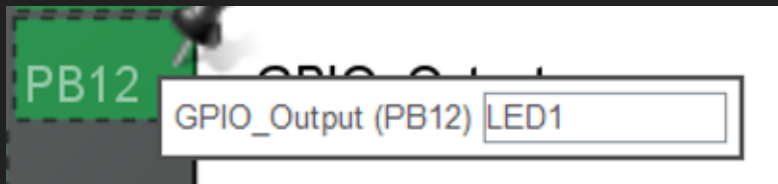
```
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_SET);
```

变成：

```
HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_PIN, GPIO_SET);
```

GPIOB -> LED1_GPIO_Port

GPIO_PIN_12 -> LED1_PIN



反转引脚状态

```
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
```

高电平 -> 低电平

低电平 -> 高电平

获取时间函数

```
void HAL_GetTick();
```

获取自程序启动到现在经过了多少ms

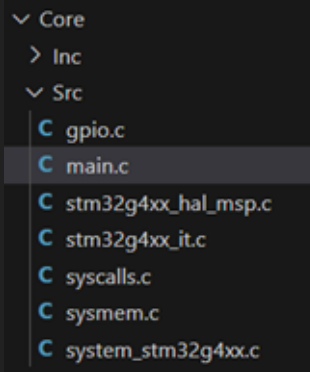
阻塞程序函数

```
void HAL_Delay(uint32_t Delay)
```

停止运行你写的程序 Delay 个毫秒

程序运行位置

- Core/Src/main.c
- `int main(void)`
- `while (1)`



```
int main(void)
{
    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {

        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}
```

代码写在 /* USER CODE BEGIN 3 */ 下面 ! ! !

```
while (1)
```

```
{
```

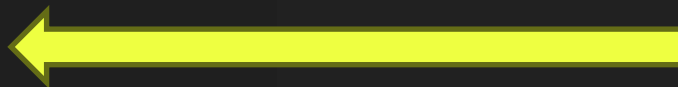
```
/* USER CODE END WHILE */
```

```
不要写在这 ! ! !
```

```
/* USER CODE BEGIN 3 */
```

```
写在这里 ! ! !
```

```
}
```



While循环里一定要加HAL_Delay() ! ! !

```
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    HAL_Delay(1);
}
```

Task1: 写一个程序, 让小灯以500ms的间隔亮灭

Practice Time



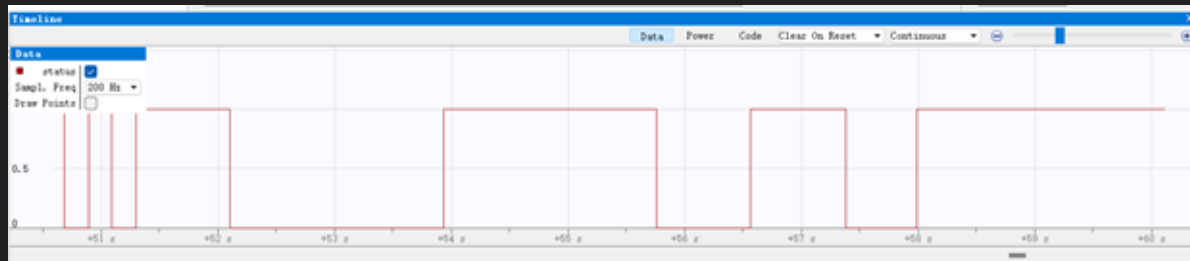
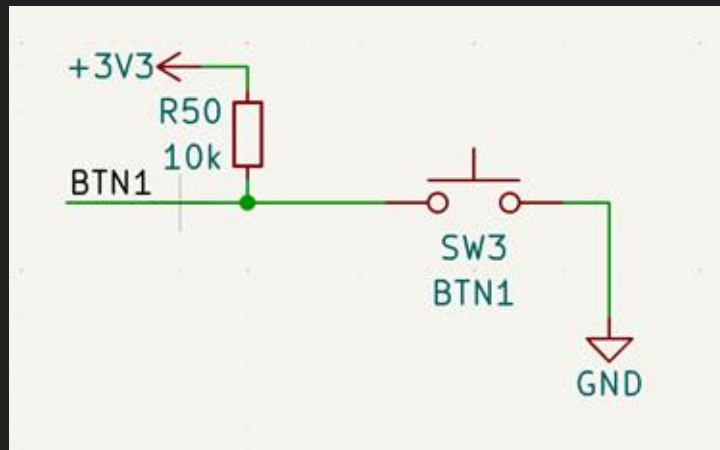
Task1: 写一个程序, 让小灯以500ms的间隔亮灭

```
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    HAL_GPIO_WritePin(LED1_GPIO_Port,LED1_Pin,GPIO_PIN_SET);
    HAL_Delay(500);
    HAL_GPIO_WritePin(LED1_GPIO_Port,LED1_Pin,GPIO_PIN_RESET);
    HAL_Delay(500);
}
```


按钮

- 按钮按下后, 电路接通, BTN1读到的电压为R38. 因为分压低所以读到的为低电压(0).
- 按钮松开时, 电路断开, 读到的为3V3,高电压(1).
- 可以使用HAL_GPIO_ReadPin来读取状态



GPIO 上拉/下拉

- 如果不打开上拉或者下拉的话(如果没有硬件上拉或者下拉), GPIO会处于**悬空状态**, 此时读到的电压是不确定的.
- 打开**上拉**的话, 会相当于用一个电阻拉高电压, 读到的**默认电压**会变成**高电压**.
- 打开**下拉**的话, 会相当于用一个电阻拉低电压, 读到的**默认电压**会变成**低电压**.
- 我们现在用的开发板存在**硬件上拉**, 所以不需要考虑这个.



使用Ozone Debug - Watched Data

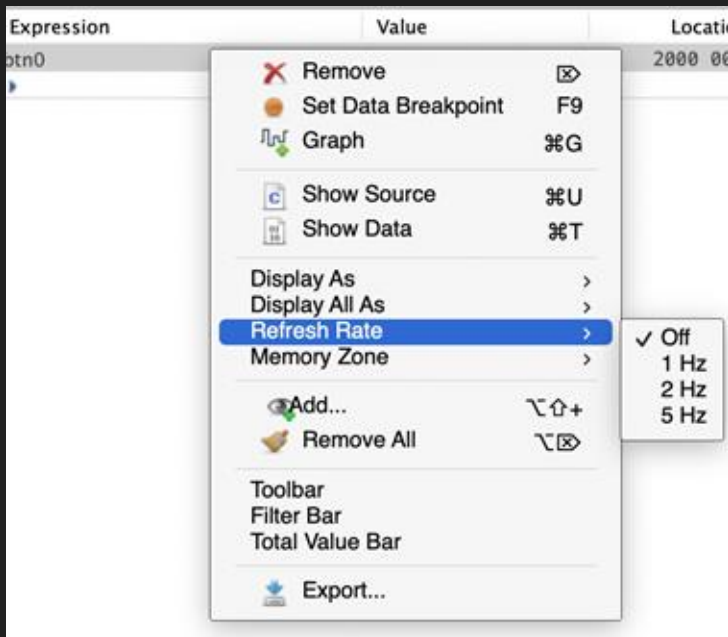
Watched Data 1

Expression	Value	Location	Refresh
btn0	0 ('\0')	2000 0029	Off

- 如果想要观测数据, 双击红框内的区域, 输入你想要观测的变量名.
- 如果显示 “<outofscope>”, 尝试加上 **volatile** 或者 **static** 在变量声明的签名.
例如 “**volatile uint8_t btn0;**”

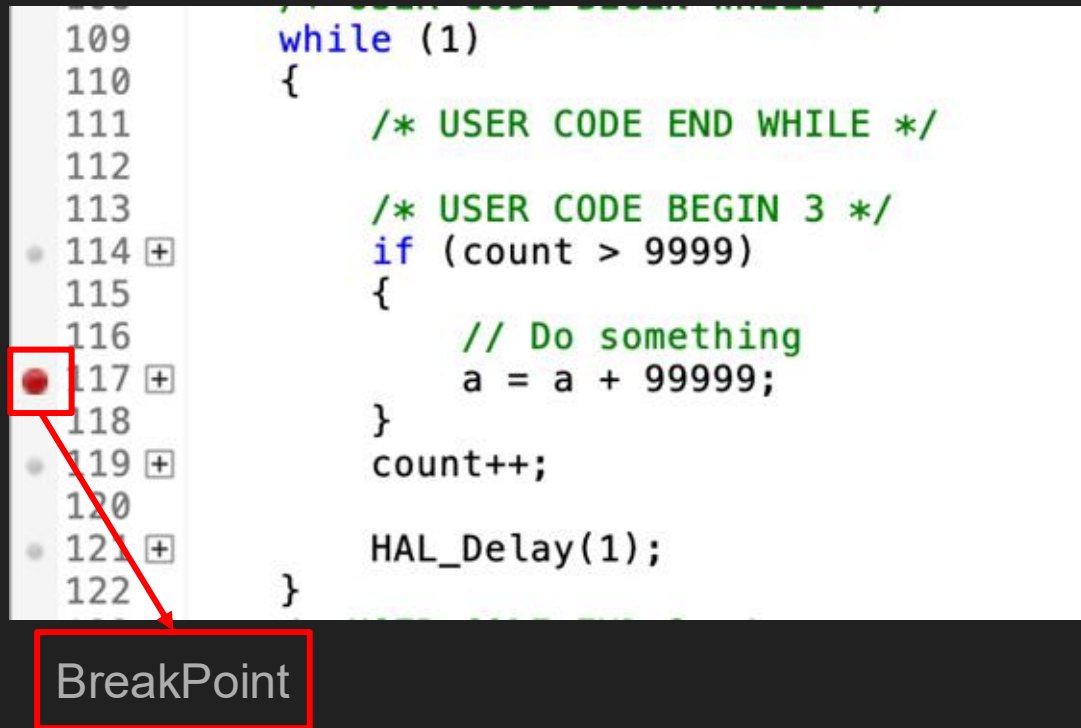
使用Ozone进行Debug - Watched Data Refresh Rate

- 如果要让数据刷新, 右键点击变量, 选择 **refresh rate** 然后选择其他频率.
- 同时可以通过更改**Display All As**改变变量显示形式 (二进制, 十六进制)

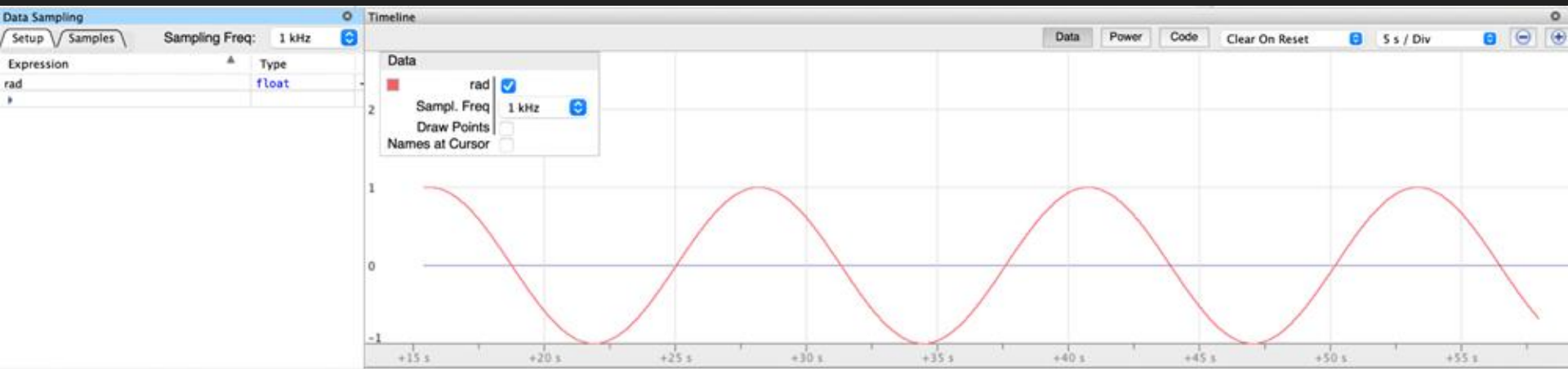


使用Ozone进行Debug – 断点

- 当代码运行到**断点**处时会**暂停**
- 可以使用**断点**来缩小查找范围



使用Ozone进行Debug - Data Sampling & Timeline



- 右键要查看的数据-graph
- 可视化数据变化

Task2: 写一个程序，按钮按下时灯亮，松开时灭

Task3: 写一个程序，按钮按一下灯亮，再按一下灯灭

Practice Time

Try your best!!!



Task2: 写一个程序，按钮按下时灯亮，松开时灭

```
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    GPIO_PinState button_state = HAL_GPIO_ReadPin(button_GPIO_Port,button_Pin);
    if (button_state == GPIO_PIN_RESET){
        HAL_GPIO_WritePin(LED1_GPIO_Port,LED1_Pin,GPIO_PIN_SET);
    }
    else
    {
        HAL_GPIO_WritePin(LED1_GPIO_Port,LED1_Pin,GPIO_PIN_RESET);
    }
    HAL_Delay(1);
}
```

Task3: 写一个程序，按钮按一下灯亮，再按一下灯灭

```
GPIO_PinState current_btn_state = HAL_GPIO_ReadPin(button_GPIO_Port,button_Pin);
GPIO_PinState last_btn_state;

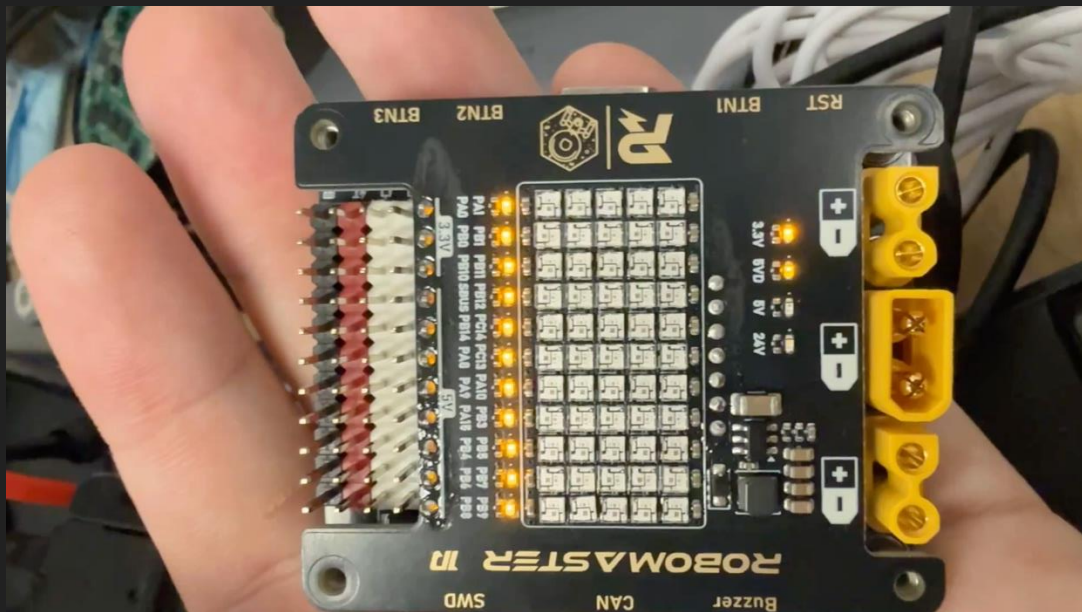
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    last_btn_state = current_btn_state;
    current_btn_state = HAL_GPIO_ReadPin(button_GPIO_Port,button_Pin);
    bool trigger = (current_btn_state == GPIO_PIN_RESET) && (last_btn_state == GPIO_PIN_SET);
    if (trigger)
    {
        HAL_GPIO_TogglePin(LED1_GPIO_Port,LED1_Pin);
    }

    HAL_Delay(1);
}
```

Task4 实现如下效果



如何让代码更简洁？